

Meeting Summary – App Prototype for Receipt Processing

Date: 12.03.2026

Ort: Teams

Topic: First meeting

participants: Lorenz, Raphael, Mustafa Arian, Christof Feischl (AI-developer), Jürgen Palkoska (Head of SoftwareDev)

Contents

1. App Mockup & Technology	2
2. App Structure	2
3. Source Code Documentation	2
4. Sprint Review / Preview	3
5. Language:	3
6. Data Sources:	3
7. Implementation Approach	4
Main Focus	4
TASKS	6
GOAL – 1 SPRINT:	6
TEAM:	6

IMPORTANT NOTES

- **NEXT MEETING – 8.4.2026 14:00**
 - **Communication -> Email**
 - **ONLY DO FIRST TASK (Document classification and extraction)**
-

1. App Mockup & Technology

- For visualization/UI → PoC, **Flutter is suitable** and can be modeled similar to a BMI-style interface (**UI infos from screenshots**).
 - From a technical perspective → all used Models/Methods should also be **compatible for .NET MAUI** (their environment)
- Current plan: Android only
- Latency:
 - 0.1 sec → instant
 - 1 sec → MAXIMUM
- Storage - Re: maximum model sizes, etc.:
 - As a starting point for research: possibly Qwen-3.5 with 0.8B and quantized. Or, for example, MobileBERT.

2. App Structure

We will get a demo from them how their app looks like → same type.

Main Use Case

1. Take a photo of a receipt.
2. Process the image.
3. Extract structured data (e.g., UID, amounts, etc.).
4. Classify item and extract main infos
5. Send this structured data to the backend
 - In this case as e.g. json → then visualize it
6. The FIBU system acts as the backend in their app → for our use just visualize
7. Document processing should occur on the device!

Additional add on:

- Summarize the content on receipt

3. Source Code Documentation

Decision / Recommendation:

- Use **comments** and **docstrings** within the code.
- At the end, **summarize** important things in a **Word document**.

Documentation should include

- Basic **setup** instructions
 - Installation steps
 - Order of execution

- Environment setup
 - Required systems
 - Testing approach
 - Which IDE to use
- **Goal:** allow someone to start the project quickly.

Concept Documentation

- Explanation of:
 - The concept of the system
 - Technologies used
 - Testing approach
 - Main outcome
 - Final conclusion – FAZIT

GITHUB:

Repository access: invite in the end

4. Sprint Review / Preview

- Deliver a **working version** of the app after each sprint
- And present progress updates during sprint reviews
- Define goals which we want to reach

5. Language:

App Language

- Code, comments can be in English
- Should be mainly compatible with German input (also English)
- UI can be English or German (maybe language selector)

6. Data Sources:

- Kaggel Dataset links (get from them)
- Search for some for our self
- LIST of types:
 - Invoices
 - Receipts
 - confirmation of leave for caregiving
 - changes to master data (registration forms, marriage certificates)
 - (for ESS)

- delivery notes
- doctor's notes
- (Rechnungen, Kassenbelege, Bestätigung für Pflegefreistellungen, Stammdatenänderung (Meldezettel, Heiratsurkunde) (für ESS), Lieferscheine, Arztbestätigungen
- Maybe we get some demo invoices/recipes

INVOICES

[Inv3D: a high-resolution 3D invoice dataset for template-guided single-image document unwarping](#), [naver-clova-ix/cord-v1 · Datasets at Hugging Face](#), [Francisco-Cruz/InvoicesReceiptsPT · Datasets at Hugging Face](#)

7. Implementation Approach

Training

- Generate/search synthetic data
- Conduct own tests with the generated data

Output

- Results should be generated as a JSON structure
- Additionally provide a visual representation of the extracted data
- Test it with real / good examples

Main Focus

Recommended Models:

1. OCR -> feature extraction
 2. Vision LM
 3. Classic ML
 4. Finetune
- 1. Document classification => automatic detection of whether the document is an invoice, a receipt (or the type of document), a doctor's note, etc.
 - 2. Extract key fields; examples:
 - Invoice amount for invoices and expense receipts
 - Date and time for doctor's time confirmation
 - Expense receipt: Date
 - etc.
 - If there is still room in the project, additional use cases, e.g.

- 3. Splitting a PDF document containing multiple delivery notes and invoices into individual receipts
- 4. “Document content summary” on device

TASKS

- Uv sync - GITHUB CLEAN
- Mobile App
 - UI
 - Logic
- Models
 - Find models
 - Train / test
 - Fine tune
 - Convert to mobile on Edge
- DATA
 - Produce
 - Get
 - Label

GOAL – 1 SPRINT (08.04):

- Model structure (which we use)
- Datasets covering all types
- PoC for the Mobile App (not including model)

WORK-distribution:

Natalie: research convertibility to mobile, OCR Model

Raphael + Klevi: search model + fine tune / test -> convert it

Lorenz + Celina: Structure/UI/Logic of Mobile-APP (optionally search for additional model)